

Лексичний аналіз методом діаграми станів

Мета роботи — проектування та програмна реалізація лексичного аналізатора мови програмування Krok з використанням діаграми станів.

1. Коротка характеристика мови

Krok — проста імперативна процедурна мова програмування загального призначення, статично і суворо типізована; мова-прототип — Go, рівень складності 2. Програма складається з одного файлу: оголошення пакета `package main`, імпорту `import "fmt"` та оголошень змінних, констант, типів і функцій; виконання починається з функції `main`. Мова має типи `int`, `float`, `bool`, `string`, користувацькі скалярні типи, масиви та структури; інструкції присвоєння, розгалуження `if / else if / else`, циклу `for` (з передумовою та трикомпонентного), введення й виведення `fmt.Scan`, `fmt.Print`, `fmt.Println`. Як і в мові Go, крапку з комою наприкінці рядка вставляє лексичний аналізатор.

2. Лексична структура

Алфавіт: латинські літери, символ підкреслення, цифри та спеціальні знаки; усередині рядкових літералів і коментарів допустимі будь-які символи Unicode, крім символів кінця рядка. Мова чутлива до регістру.

Пробільні символи (пробіл, табуляція) і роздільник рядків лише розділяють лексеми. Однорядковий коментар починається з `//` і триває до кінця рядка.

Ідентифікатор: `Letter { Letter | Digit }`. Лексема, що збігається з ключовим словом, є ключовим словом; `true` і `false` — логічні літерали.

Літерали: цілі (без ведучих нулів), дійсні (з дробовою частиною та/або експонентою, цілу чи дробову частину можна пропустити), рядкові (у подвійних лапках, з escape-послідовностями `\n \t \r \" \\`), логічні.

Лексеми виділяються за правилом найдовшого збігу: `**`, `<=`, `:=`, `++` — одна лексема.

Автоматична вставка `;`: наприкінці рядка або файлу вставляється `;`, якщо остання лексема рядка — ідентифікатор, літерал, одне зі слів `return`, `int`, `float`, `bool`, `string`, оператор `++/--` або дужка `)`, `]`, `}`.

Таблиця 1 — Таблиця лексем мови Krok

Код	Приклади лексем	Токен	Клас	Неформальний опис
1	<code>x</code> , <code>total</code> , <code>Point</code> , <code>tmp1</code>	<code>id</code>	IDN	ідентифікатор
2	<code>0</code> , <code>42</code> , <code>1000</code>	<code>intnum</code>	CON	ціле без знака
3	<code>3.14</code> , <code>2.</code> , <code>.5</code> , <code>1e3</code> , <code>6.02E+23</code>	<code>floatnum</code>	CON	дійсне без знака
4	<code>""</code> , <code>"Krok"</code> , <code>"a\tb\n"</code>	<code>strlit</code>	CON	рядковий літерал
5	<code>true</code>	<code>boolval</code>	CON	логічне значення
6	<code>false</code>	<code>boolval</code>	CON	логічне значення
7	<code>package</code>	<code>keyword</code>	TRM	символ <code>package</code>
8	<code>import</code>	<code>keyword</code>	TRM	символ <code>import</code>
9	<code>func</code>	<code>keyword</code>	TRM	символ <code>func</code>
10	<code>var</code>	<code>keyword</code>	TRM	символ <code>var</code>
11	<code>const</code>	<code>keyword</code>	TRM	символ <code>const</code>
12	<code>type</code>	<code>keyword</code>	TRM	символ <code>type</code>
13	<code>struct</code>	<code>keyword</code>	TRM	символ <code>struct</code>
14	<code>if</code>	<code>keyword</code>	TRM	символ <code>if</code>
15	<code>else</code>	<code>keyword</code>	TRM	символ <code>else</code>
16	<code>for</code>	<code>keyword</code>	TRM	символ <code>for</code>
17	<code>return</code>	<code>keyword</code>	TRM	символ <code>return</code>
18	<code>int</code>	<code>keyword</code>	TRM	символ <code>int</code>
19	<code>float</code>	<code>keyword</code>	TRM	символ <code>float</code>
20	<code>bool</code>	<code>keyword</code>	TRM	символ <code>bool</code>
21	<code>string</code>	<code>keyword</code>	TRM	символ <code>string</code>

22	len	keyword	TRM	символ len
23	fmt	keyword	TRM	символ fmt
24	Print	keyword	TRM	символ Print
25	Println	keyword	TRM	символ Println
26	Scan	keyword	TRM	символ Scan
27	+	add op	TRM	додавання, унарний плюс, конкатенація
28	-	add op	TRM	віднімання, унарний мінус
29	*	mult op	TRM	множення; розіменування
30	/	mult op	TRM	ділення
31	%	mult op	TRM	остача від ділення
32	**	pow op	TRM	піднесення до степеня
33	==	rel op	TRM	дорівнює
34	!=	rel op	TRM	не дорівнює
35	<	rel op	TRM	менше
36	<=	rel op	TRM	менше або дорівнює
37	>	rel op	TRM	більше
38	>=	rel op	TRM	більше або дорівнює
39	&&	and op	TRM	логічне «і»
40		or op	TRM	логічне «або»
41	!	not op	TRM	логічне заперечення
42	=	assign op	TRM	присвоєння
43	:=	define op	TRM	оголошення з ініціалізацією
44	++	incdec op	TRM	інкремент
45	--	incdec op	TRM	декремент
46	&	addr op	TRM	взяття адреси
47	(	paren	TRM	ліва кругла дужка
48	)	paren	TRM	права кругла дужка
49	[	bracket	TRM	ліва квадратна дужка
50	]	bracket	TRM	права квадратна дужка
51	{	brace	TRM	ліва фігурна дужка
52	}	brace	TRM	права фігурна дужка
53	,	punct	TRM	кома
54	;	punct	TRM	крапка з комою (явна або вставлена)
55	.	punct	TRM	крапка (доступ до поля, fmt.)
56	// коментар	comment	—	однорядковий коментар (ігнорується)
57	\32	ws	—	пробіл (ігнорується)
58	\t	ws	—	горизонтальна табуляція (ігнорується)
59	\n, \r\n	eol	—	кінець рядка (ігнорується; може породити ;)

3. Частина граматики мови, необхідна для лексичного аналізу

```

SourceText    = { Lexeme }
Lexeme        = Token | Comment | WhiteSpace | EndOfLine
Token         = Ident | Keyword | IntLit | FloatLit | StringLit | BoolLit
              | Operator | Delimiter
Keyword       = package | import | func | var | const | type | struct
              | if | else | for | return | int | float | bool | string
              | len | fmt | 'Print' | 'Println' | 'Scan'
Operator      = ArithOp | RelOp | LogicOp | AssignOp | IncDecOp | '&'
ArithOp       = '+' | '-' | '*' | '/' | '%' | '**'
LogicOp       = '&&' | '||' | '!'
Delimiter     = '(' | ')' | '[' | ']' | '{' | '}' | ',' | ';' | '.'
Ident         = Letter { Letter | Digit }
IntLit        = '0' | NonZeroDigit { Digit }
FloatLit      = Digits '.' [ Digits ] [ Exponent ]
              | Digits Exponent
              | '.' Digits [ Exponent ]
Digits        = Digit { Digit }
Exponent      = ( 'e' | 'E' ) [ '+' | '-' ] Digits
StringLit     = '"' { StringChar } '"'
StringChar    = OrdinaryChar | EscapeSeq
EscapeSeq     = '\\' ( 'n' | 't' | 'r' | '"' | '\'' )
OrdinaryChar  = ? будь-який символ Unicode, крім '"', '\', '\n' та '\r' ?
Comment       = '//' { CommentChar }
CommentChar   = ? будь-який символ Unicode, крім '\n' та '\r' ?
Alphabet       = Letter | Digit | SpecialChar | WhiteSpace | EndOfLine | OtherChar
Letter        = LowerLetter | UpperLetter | '_'
LowerLetter   = 'a' | 'b' | 'c' | 'd' | 'e' | 'f' | 'g' | 'h' | 'i'

```


5. Діаграма станів (символьне представлення)

Діаграма станів як детермінований скінченний автомат: $M = (Q, \Sigma, \delta, q_0, F)$.

Множина станів: $Q = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 101, 102, 103, 104, 105, 106, 107, 108\}$.

Алфавіт: $\Sigma = \text{Letter} \cup \text{Digit} \cup \{ . \text{ " } \backslash + - * / \% = ! < > \& | : ; , () [] \{ \} \text{ пробіл } \backslash t \backslash r \backslash n \} \cup \text{OtherChar}$, де OtherChar — символи Unicode поза алфавітом мови, допустимі лише в рядкових літералах і коментарях.

Класи символів: Letter — латинські літери та `_`; Digit — цифри; dot — крапка; ws — пробіл, табуляція, повернення каретки; nl — новий рядок; quote — подвійні лапки; bs slash — зворотна коса риска; кожен зі знаків `+ - * / \% = ! < > \& | : ; , ( ) [ ] \{ \}` утворює власний клас; other — символи, що не належать до поточної лексеми. Переходи за окремими символами `0, e, E, n, t, r` мають пріоритет над переходами за класом.

Функція переходів:

```

δ(0, Letter) = 1; δ(0, 0) = 3; δ(0, Digit) = 4; δ(0, dot) = 11; δ(0, quote) = 14; δ(0, /) = 17; δ(0, +) = 20; δ(0, -) = 21; δ(0, *) = 22; δ(0, =) = 23; δ(0, !) = 24; δ(0, <) = 25; δ(0, >) = 26; δ(0, &) = 27; δ(0, |) = 28; δ(0, :) = 29; δ(0, %) = 30; δ(0, () = 30; δ(0, )) = 30; δ(0, []) = 30; δ(0, [] = 30; δ(0, {} = 30; δ(0, }) = 30; δ(0, ,) = 30; δ(0, ;) = 30; δ(0, ws) = 0; δ(0, nl) = 32; δ(0, other) = 101;
δ(1, Letter) = 1; δ(1, Digit) = 1; δ(1, other) = 2;
δ(3, Digit) = 5; δ(3, dot) = 6; δ(3, e) = 8; δ(3, E) = 8; δ(3, Letter) = 104; δ(3, other) = 12;
δ(4, Digit) = 4; δ(4, dot) = 6; δ(4, e) = 8; δ(4, E) = 8; δ(4, Letter) = 104; δ(4, other) = 12;
δ(5, Digit) = 5; δ(5, dot) = 6; δ(5, e) = 8; δ(5, E) = 8; δ(5, Letter) = 104; δ(5, other) = 102;
δ(6, Digit) = 7; δ(6, e) = 8; δ(6, E) = 8; δ(6, Letter) = 104; δ(6, other) = 13;
δ(7, Digit) = 7; δ(7, e) = 8; δ(7, E) = 8; δ(7, Letter) = 104; δ(7, other) = 13;
δ(8, Digit) = 10; δ(8, +) = 9; δ(8, -) = 9; δ(8, other) = 103;
δ(9, Digit) = 10; δ(9, other) = 103;
δ(10, Digit) = 10; δ(10, Letter) = 104; δ(10, other) = 13;
δ(11, Digit) = 7; δ(11, other) = 31;
δ(14, quote) = 16; δ(14, bs slash) = 15; δ(14, nl) = 105; δ(14, other) = 14;
δ(15, n) = 14; δ(15, t) = 14; δ(15, r) = 14; δ(15, quote) = 14; δ(15, bs slash) = 14; δ(15, other) = 106;
δ(17, /) = 18; δ(17, other) = 31;
δ(18, nl) = 19; δ(18, other) = 18;
δ(20, +) = 30; δ(20, other) = 31;
δ(21, -) = 30; δ(21, other) = 31;
δ(22, *) = 30; δ(22, other) = 31;
δ(23, =) = 30; δ(23, other) = 31;
δ(24, =) = 30; δ(24, other) = 31;
δ(25, =) = 30; δ(25, other) = 31;
δ(26, =) = 30; δ(26, other) = 31;
δ(27, &) = 30; δ(27, other) = 31;
δ(28, |) = 30; δ(28, other) = 108;
δ(29, =) = 30; δ(29, other) = 107;

```

Стартовий стан: $q_0 = 0$.

Множина заключних станів: $F = \{2, 12, 13, 16, 19, 30, 31, 32, 101, 102, 103, 104, 105, 106, 107, 108\}$.

Заклучні стани, що потребують додаткової обробки: $F_{\text{star}} = \{2, 12, 13, 19, 31\}$; $\text{FERROR} = \{101, 102, 103, 104, 105, 106, 107, 108\}$.

6. Семантичні процедури

Таблиця 2 — Семантичні процедури

Стан	Токен	Семантичні процедури
2	id, keyword або boolval	Визначити: id, keyword чи boolval? Якщо id — обробити таблицю ідентифікаторів, якщо boolval — таблицю констант. Занести лексему в таблицю розбору. Повернути необроблений символ у вхідний потік. Перейти у стартовий стан
12	intnum	Обробити таблицю констант. Занести лексему в таблицю розбору. Повернути необроблений символ у вхідний потік. Перейти у стартовий стан
13	floatnum	Те саме, що й для стану 12

16	strlit	Додати закриваючі лапки до лексеми. Обробити таблицю констант. Занести лексему в таблицю розбору. Перейти у стартовий стан
19	(коментар)	Відкинути лексему. Повернути символ нового рядка у вхідний потік. Перейти у стартовий стан
30	оператори та роздільники	Додати останній символ до лексеми. Занести лексему в таблицю розбору. Перейти у стартовий стан
31	оператори та роздільники	Занести лексему в таблицю розбору. Повернути необроблений символ у вхідний потік. Перейти у стартовий стан
32	eol	Якщо остання лексема рядка вимагає цього — занести в таблицю розбору лексему ; . Збільшити лічильник рядків на 1. Перейти у стартовий стан
101–108	—	Вивести діагностичне повідомлення з номером рядка. Аварійно завершити роботу з кодом, що дорівнює номеру стану

7. Структури даних і програмна реалізація

Лексичний аналізатор `krok_lex.py` написано мовою Python за зразком `my_lang_lex.py`: збережено словники `stf`, `tokenTable`, `tokStateTable`, множини `F`, `Fstar`, `Error`, таблиці `tableOfSymb`, `tableOfId`, `tableOfConst` та функції `lex`, `nextChar`, `putCharBack`, `nextState`, `processing`, `getToken`, `indexIdConst`, `fail`. Суттєві відмінності такі.

1. Функція `nextState(state, classCh, char)` шукає перехід спершу за конкретним символом, потім за класом символу, і лише потім — за класом `other`. Це потрібно, бо `e` та `E` в експоненті водночас є літерами, а `0` на початку числа — цифрою.

2. Автоматична вставка `;` реалізована як семантична процедура стану 32: змінна `lastRec` зберігає токен і лексему останнього запису поточного рядка. Кінець файлу трактується як кінець рядка — до тексту програми дописується символ нового рядка.

3. Таблиця констант містить не лише числові, а й рядкові та логічні літерали: `{ Const : (token, idxConst) }`.

4. Коментар має власний заключний стан 19 із зірочкою: лексема відкидається, а символ нового рядка повертається у потік, щоб його обробив стан 32.

5. Запис у таблицю розбору винесено у функцію `addRecord`, а запуск аналізу — у функцію `analyze(text)`, яка очищує таблиці та лічильники; це дозволяє викликати аналізатор багаторазово під час тестування.

8. Приклади програм для тестування

Базовий приклад містить усі лексеми з таблиці лексем мови (файл `tests/base_example.krok`):

```

1 // Базовий приклад програми мовою Krok:
2 // містить приклади всіх лексем із таблиці токенів.
3 package main
4
5 import "fmt"
6
7 const Size int = 5 // іменована константа з явним типом
8 const Eps = 1.5e-3 // тип float визначається за літералом
9 const Title = "Krok:\t\"крк\" \\ 2026"
10
11 type Celsius float // користувацький скалярний тип
12
13 type Point struct {
14     x, y float
15     label string
16 }
17
18 var total int // глобальна змінна, початкове значення 0
19 var ready bool = false // глобальна змінна з ініціалізацією
20
21 // Рекурсивне обчислення факторіала.
```

```

22 func factorial(n int) int {
23     if n <= 1 {
24         return 1
25     }
26     return n * factorial(n-1)
27 }
28
29 // Передача параметрів за посиланням.
30 func swap(a, b *int) {
31     t := *a
32     *a = *b
33     *b = t
34 }
35
36 // Масив передається за значенням (копіюється).
37 func sum(v [Size]int) int {
38     s := 0
39     for i := 0; i < len(v); i++ {
40         s = s + v[i]
41     }
42     return s
43 }
44
45 func distance(p Point) float {
46     return (p.x**2 + p.y**2) **.5
47 }
48
49 func main() {
50     var n, m int
51     fmt.Scan(&n, &m)
52     swap(&n, &m)
53     fmt.Println("n =", n, "m =", m, "n! =", factorial(n%10))
54
55     var data [Size]int = [Size]int{3, -1, 4, 1, +5}
56     var grid [2][3]float
57     grid[1][2] = 2.
58     total = sum(data)
59
60     var t Celsius = 36.6
61     t = t + Celsius(n)
62     x := float(total)/2 - 7*3 // int неявно розширюється до float
63     k := int(-x)
64
65     var p Point
66     p.x = 3.0
67     p.y = 4E0
68     p.label = ""
69     d := distance(p)
70
71     ok := true
72     ready = d >= 5.0 && !(k == 0) || total != 12
73     for k > 10 {
74         k--
75     }
76     for i := 0; i < 3; i++ {
77         if grid[1][2] > float(i) {
78             fmt.Print(i, " ")
79         } else if i == 2 {
80             fmt.Print("last")
81         } else {
82             fmt.Print("-")
83         }
84     }
85     fmt.Println()
86     fmt.Print(Title, "\n")
87     fmt.Println(t, d, ready, ok, Eps, x < -1.0, 2**3**2, k)
88 }

```

Приклад із вкладеними конструкціями (файл tests/nested.krok):

```

package main

import "fmt"

func main() {
    var m [2][3]int = [2][3]int{{1, 2, 3}, {4, 5, 6}}

```

```

for i := 0; i < 2; i++ {
    for j := 0; j < 3; j++ {
        if m[i][j]%2 == 0 {
            if m[i][j] > 4 {
                fmt.Println(m[i][j], "велике парне")
            } else {
                fmt.Print(m[i][j], " ")
            }
        }
    }
}
}
}

```

9. План тестування

Таблиця 3 — План тестування

№	Тип випробування	Очікуваний результат	Кількість
1	базовий приклад	успішне виконання, таблиці розбору, ідентифікаторів, констант	1
2	вкладені конструкції	успішне виконання, таблиці ідентифікаторів, констант	1
3	наявність недопустимих лексем	повідомлення про помилку з номером рядка, діагностика, аварійне завершення з кодом стану	8
4	надлишок чи відсутність синтаксично виправданих лексем	успішне виконання: лексичних помилок немає	1
5	помилково написані ключові слова	успішне виконання: слова розпізнано як ідентифікатори	1
6	особливості лексики Крок: найдовший збіг, вставка ;, коментарі	успішне виконання, правильний поділ на лексеми	2

Додатково програма tests/run_tests.py автоматично порівнює потік лексем цього аналізатора з еталонним лексером специфікації мови на дев'яти прикладах програм і перевіряє 18 окремих ситуацій (усього 27 перевірок).

10. Протокол тестування

Тест № 1.1. Базовий приклад (текст програми — у розд. 8).

```

>python krok_lex.py test_1_1.krok
3  package      keyword
3  main         id          1
3  ;            punct      <- вставлено автоматично
5  import       keyword
5  "fmt"        strlit     1
5  ;            punct      <- вставлено автоматично
7  const        keyword
7  Size         id          2
7  int          keyword
7  =            assign_op
7  5            intnum     2
7  ;            punct      <- вставлено автоматично
8  const        keyword
8  Eps          id          3
8  =            assign_op
8  1.5e-3       floatnum   3
8  ;            punct      <- вставлено автоматично
9  const        keyword
9  Title        id          4
9  =            assign_op
9  "Krok:\t\"кр\ "  strlit     4
9  ;            punct      <- вставлено автоматично
11 type        keyword
11 Celsius      id          5
11 float        keyword
11 ;            punct      <- вставлено автоматично
... (463 рядків пропущено) ...
88 ;            punct      <- вставлено автоматично
Lexer: Лексичний аналіз завершено успішно
-----

```

```
tableOfId:{'main': 1, 'Size': 2, 'Eps': 3, 'Title': 4, 'Celsius': 5, 'Point': 6, 'x': 7, 'y': 8, 'label': 9, 'total': 10, 'ready': 11, 'factorial': 12, 'n': 13, 'swap': 14, 'a': 15, 'b': 16, 't': 17, 'sum': 18, 'v': 19, 's': 20, 'i': 21, 'distance': 22, 'p': 23, 'm': 24, 'data': 25, 'grid': 26, 'k': 27, 'd': 28, 'ok': 29}
tableOfConst:{'fmt': ('strlit', 1), '5': ('intnum', 2), '1.5e-3': ('floatnum', 3), '"Krok:\\t\\"крок\\"' '\\\\ 2026"' ('strlit', 4), 'false': ('boolval', 5), '1': ('intnum', 6), '0': ('intnum', 7), '2': ('intnum', 8), '.5': ('floatnum', 9), '"n="' ('strlit', 10), '"m="' ('strlit', 11), '"n!="' ('strlit', 12), '10': ('intnum', 13), '3': ('intnum', 14), '4': ('intnum', 15), '2.': ('floatnum', 16), '36.6': ('floatnum', 17), '7': ('intnum', 18), '3.0': ('floatnum', 19), '4E0': ('floatnum', 20), '""': ('strlit', 21), 'true': ('boolval', 22), '5.0': ('floatnum', 23), '12': ('intnum', 24), '" "' ('strlit', 25), '"last"' ('strlit', 26), '"-"' ('strlit', 27), '"\\n"' ('strlit', 28), '1.0': ('floatnum', 29)}
```

Коментар до тесту № 1.1: усі лексеми розпізнано, токени визначено правильно; у таблицю розбору потрапили й автоматично вставлені ;. Аналіз завершено успішно.

Тест № 2.1. Вкладені конструкції (текст програми — у розд. 8).

```
>python krok_lex.py test_2_1.krok
1 package keyword
1 main id 1
1 ; punct <- вставлено автоматично
3 import keyword
3 "fmt" strlit 1
3 ; punct <- вставлено автоматично
5 func keyword
5 main id 1
5 ( paren
5 ) paren
5 { brace
6 var keyword
6 m id 2
6 [ bracket
6 2 intnum 2
6 ] bracket
6 [ bracket
6 3 intnum 3
6 ] bracket
6 int keyword
6 = assign_op
6 [ bracket
... (114 рядків пропущено) ...
18 ; punct <- вставлено автоматично
Lexer: Лексичний аналіз завершено успішно
-----
tableOfId:{'main': 1, 'm': 2, 'i': 3, 'j': 4}
tableOfConst:{'fmt': ('strlit', 1), '2': ('intnum', 2), '3': ('intnum', 3), '1': ('intnum', 4), '4': ('intnum', 5), '5': ('intnum', 6), '6': ('intnum', 7), '0': ('intnum', 8), '"велике парне"' ('strlit', 9), '" "' ('strlit', 10)}
```

Коментар до тесту № 2.1: вкладені цикли, розгалуження та вкладений літерал масиву розібрано правильно; двосимвольні оператори :=, ++, == виділено як одну лексему.

Тест № 3.1. Недопустимий символ.

```
1 x := 5 @ 3

>python krok_lex.py test_3_1.krok
1 x id 1
1 := define_op
1 5 intnum 1
1
Lexer: у рядку 1 неочікуваний символ @
Lexer: Аварійне завершення програми з кодом 101
-----
tableOfId:{'x': 1}
tableOfConst:{'5': ('intnum', 1)}
```

Коментар до тесту № 3.1: символ @ не належить алфавіту — стан 101.

Тест № 3.2. Цілий літерал із ведучим нулем.

```
1 x := 007

>python krok_lex.py test_3_2.krok
```

```
1 x id 1
1 := define_op
1
Lexer: у рядку 1 цілий літерал із ведучим нулем: 007
Lexer: Аварійне завершення програми з кодом 102
-----
tableOfId: {'x': 1}
tableOfConst: {}
```

Коментар до тесту № 3.2: стан 102; запис 007.5 при цьому допустимий.

Тест № 3.3. Неправильна експонента.

```
1 x := 1e+
```

```
>python krok_lex.py test_3_3.krok
1 x id 1
1 := define_op
1
Lexer: у рядку 1 неправильна експонента дійсного літерала: 1e+
Lexer: Аварійне завершення програми з кодом 103
-----
tableOfId: {'x': 1}
tableOfConst: {}
```

Коментар до тесту № 3.3: після знака експоненти немає цифр — стан 103.

Тест № 3.4. Цифра перед літерою.

```
1 x := 12abc
```

```
>python krok_lex.py test_3_4.krok
1 x id 1
1 := define_op
1
Lexer: у рядку 1 ідентифікатор не може починатися з цифри: 12a
Lexer: Аварійне завершення програми з кодом 104
-----
tableOfId: {'x': 1}
tableOfConst: {}
```

Коментар до тесту № 3.4: ідентифікатор не може починатися з цифри — стан 104.

Тест № 3.5. Незакритий рядковий літерал.

```
1 s := "abc
```

```
>python krok_lex.py test_3_5.krok
1 s id 1
1 := define_op
1
Lexer: у рядку 1 незакритий рядковий літерал: "abc
Lexer: Аварійне завершення програми з кодом 105
-----
tableOfId: {'s': 1}
tableOfConst: {}
```

Коментар до тесту № 3.5: рядок не закрито до кінця рядка — стан 105.

Тест № 3.6. Невідома escape-послідовність.

```
1 s := "a\q"
```

```
>python krok_lex.py test_3_6.krok
1 s id 1
1 := define_op
1
Lexer: у рядку 1 невідома escape-послідовність \q
Lexer: Аварійне завершення програми з кодом 106
-----
tableOfId: {'s': 1}
tableOfConst: {}
```

Коментар до тесту № 3.6: послідовність \q не визначена — стан 106.

Тест № 3.7. Одиночна двокрапка.

```
1 a : b
```

```
>python krok_lex.py test_3_7.krok
1 a id 1
1
Lexer: у рядку 1 очікувався символ =, а не
Lexer: Аварійне завершення програми з кодом 107
-----
tableOfId: {'a': 1}
tableOfConst: {}
```

Коментар до тесту № 3.7: після : очікувався = — стан 107.

Тест № 3.8. Одиночна вертикальна риска.

```
1 a | b
```

```
>python krok_lex.py test_3_8.krok
1 a id 1
1
Lexer: у рядку 1 очікувався символ |, а не
Lexer: Аварійне завершення програми з кодом 108
-----
tableOfId: {'a': 1}
tableOfConst: {}
```

Коментар до тесту № 3.8: після | очікувався | — стан 108.

Тест № 4.1. Усі лексеми допустимі, але їх порядок суперечить синтаксису.

```
1 package package main
2 func ( { x = = 5 }
```

```
>python krok_lex.py test_4_1.krok
1 package keyword
1 package keyword
1 main id 1
1 ; punct <- вставлено автоматично
2 func keyword
2 ( paren
2 { brace
2 x id 2
2 = assign_op
2 = assign_op
2 5 intnum 1
2 } brace
2 ; punct <- вставлено автоматично
Lexer: Лексичний аналіз завершено успішно
-----
tableOfId: {'main': 1, 'x': 2}
tableOfConst: {'5': ('intnum', 1)}
```

Коментар до тесту № 4.1: лексичний аналізатор помилок НЕ ЗНАХОДИТЬ: синтаксичну правильність він не перевіряє.

Тест № 5.1. Помилково написані ключові слова.

```
1 fucn main() { retrun 1 }
```

```
>python krok_lex.py test_5_1.krok
1 fucn id 1
1 main id 2
1 ( paren
1 ) paren
1 { brace
1 retrun id 3
1 1 intnum 1
1 } brace
1 ; punct <- вставлено автоматично
```

```
Lexер: Лексичний аналіз завершено успішно
-----
tableOfId: {'fucn': 1, 'main': 2, 'retrun': 3}
tableOfConst: {'1': ('intnum', 1)}
```

Коментар до тесту № 5.1: fucn і retrun розпізнано як ідентифікатори; помилку виявить синтаксичний аналізатор.

Тест № 6.1. Правило найдовшого збігу.

```
1 a---b
2 x = 2**-3 <= .5
```

```
>python krok_lex.py test_6_1.krok
1 a id 1
1 -- incdec_op
1 - add_op
1 b id 2
1 ; punct <- вставлено автоматично
2 x id 3
2 = assign_op
2 2 intnum 1
2 ** pow_op
2 - add_op
2 3 intnum 2
2 <= rel_op
2 .5 floatnum 3
2 ; punct <- вставлено автоматично
Lexер: Лексичний аналіз завершено успішно
-----
tableOfId: {'a': 1, 'b': 2, 'x': 3}
tableOfConst: {'2': ('intnum', 1), '3': ('intnum', 2), '.5': ('floatnum', 3)}
```

Коментар до тесту № 6.1: a---b поділено на a, --, -, b; ** і <= — по одній лексемі; .5 — дійсний літерал.

Тест № 6.2. Коментарі та вставка крапки з комою.

```
1 x := 10 // коментар
2 s := "// не коментар"
3 if x > 0 {
4 }
```

```
>python krok_lex.py test_6_2.krok
1 x id 1
1 := define_op
1 10 intnum 1
1 ; punct <- вставлено автоматично
2 s id 2
2 := define_op
2 "// не коментар" strlit 2
2 ; punct <- вставлено автоматично
3 if keyword
3 x id 1
3 > rel_op
3 0 intnum 3
3 { brace
4 } brace
4 ; punct <- вставлено автоматично
Lexер: Лексичний аналіз завершено успішно
-----
tableOfId: {'x': 1, 's': 2}
tableOfConst: {'10': ('intnum', 1), '"// не коментар"': ('strlit', 2), '0': ('intnum', 3)}
```

Коментар до тесту № 6.2: коментар відкинуто, а ; після 10 вставлено; // у рядку — частина літерала; після { крапку з комою не вставлено, після } — вставлено.

11. Висновки

Побудовано лексичний аналізатор мови Krok методом діаграми станів: 33 робочі стани та 8 станів помилок, 104 переходи. Він розпізнає всі лексеми з таблиці лексем мови, будує таблиці розбору, ідентифікаторів і констант та локалізує лексичні помилки з точністю до рядка.

Найбільше станів знадобилося для чисел: щоб відрізнити 0, 007 (помилка) і 007.5 (дійсний літерал), довелося ввести окремі стани для нуля та для цілого з ведучим нулем. Саме тут виявилася корисною ідея переходів за окремим символом поперх переходів за класом — без неї в експоненті не відрізнити від літери.

Особливість мови Krok — автоматична вставка крапки з комою. Вона не вкладається у формалізм скінченного автомата, бо залежить від попередньої лексеми, тому реалізована семантичною процедурою стану кінця рядка.

Коректність перевірено на базовому прикладі, прикладі з вкладеними конструкціями та на прикладах із помилками, а також автоматичним порівнянням з еталонним лексером специфікації: потоки лексем збіглися на всіх дев'яти програмах. Тест 4.1 підтверджує, що лексичний аналізатор не перевіряє синтаксис: така перевірка — завдання наступної роботи практикуму.